

Experiment No.	2
Experiment Title	Spam Classification on the Spambase Dataset: Naive Bayes and k -Nearest Neighbours ¹
Student Name	Sanghamithra Namboodiri
Register Number	3122247001055
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	3rd August 2026

1 Aim and Objective

The aim of this experiment is to build, tune, and compare multiple classifiers for detecting spam email using the UCI Spambase dataset. Specifically, the objectives are to:

- Perform exploratory data analysis (EDA) on the Spambase dataset, including cleaning, deduplication, and outlier handling.
- Train and evaluate four classifiers – Gaussian Naive Bayes, Multinomial Naive Bayes, Bernoulli Naive Bayes, and k -Nearest Neighbours (KNN) – on the cleaned dataset.
- Tune the KNN hyperparameters (k , weighting scheme, and search algorithm) using both exhaustive (`GridSearchCV`) and randomized (`RandomizedSearchCV`) search, and compare their efficiency.
- Compare classifiers using Accuracy, Precision, Recall, F1-score, ROC-AUC, and 5-fold cross-validation, rather than relying on a single metric or a single train/test split.
- Analyze overfitting/underfitting and bias–variance behaviour of each model, and the computational complexity of KDTree vs. BallTree nearest-neighbour search.
- Draw conclusions about which model(s) are best suited to spam detection given the trade-offs between precision, recall, and speed.

2 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository
Number of Samples	4601 (raw); 4210 after deduplication; 809 after IQR-based outlier removal
Number of Features	57 numeric features (48 <code>word_freq_*</code> , 6 <code>char_freq_*</code> , 3 <code>capital_run_length_*</code>)
Number of Classes	2 (1 = spam, 0 = non-spam)
Missing Values	0 (before and after cleaning)
Train-Test Split	80% / 20%, stratified, <code>random_state=42</code> (plus 5-fold stratified-by-default cross-validation)

Class distribution after cleaning is imbalanced at roughly 76:24 (616 non-spam vs. 193 spam), which motivates weighting Precision, Recall, F1, and ROC-AUC more heavily than raw Accuracy throughout this experiment.

3 Preprocessing Steps

- **Missing values:** none present; no imputation required.
- **Duplicate removal:** 391 duplicate rows dropped (4601 $\rightarrow$ 4210 rows).
- **Outlier removal:** IQR method applied per column (columns with IQR = 0 skipped), reducing 4210 $\rightarrow$ 809 rows.
- **Feature scaling:** `MinMaxScaler` to $[0, 1]$, required since KNN's distance metric and BernoulliNB's binarization are both scale-sensitive.
- **Train-test split:** 80/20 stratified split (`random_state=42`).
- **Leakage prevention:** the scaler is fit *only* on the training set and reused (not refit) on the test set; for 5-fold cross-validation, the scaler is refit independently inside each fold via a `Pipeline`, so no fold's held-out data ever influences scaling.

Bug Fixes Applied

Two data-leakage issues were identified and corrected in the original pipeline:

1. **Scaling leakage:** `MinMaxScaler` was originally fit on the entire dataset *before* the train/test split, meaning the scaler's min/max statistics were computed using information from the test set. Fixed by fitting the scaler on `X_train` only, then transforming `X_test` with the already-fit scaler.
2. **Cross-validation leakage:** the 5-fold `cross_val_score` call originally reused the globally scaled features, so every fold's held-out portion had already influenced the scaling. Fixed by wrapping each model in a `Pipeline(MinMaxScaler  $\rightarrow$  model)` and passing the raw, unscaled features, so each fold scales independently.

4 Implementation Details

Environment

Python Version	3.x
Scikit-Learn Version	1.x
IDE	VS Code
Operating System	Linux
Hardware	Standard CPU

Algorithms and Theory

Naive Bayes classifiers apply Bayes' theorem under a conditional-independence assumption:

$$P(y \mid x_1, \dots, x_d) \propto P(y) \prod_{i=1}^d P(x_i \mid y) \quad (1)$$

Gaussian NB assumes $P(x_i \mid y) \sim \mathcal{N}(\mu_{i,y}, \sigma_{i,y}^2)$ (a poor fit for zero-inflated, skewed features). **Multinomial NB** models $P(x_i \mid y)$ as multinomial counts, naturally suited to word/character frequency data. **Bernoulli NB** binarizes features at a threshold (default 0.0) and applies a Bernoulli likelihood.

KNN is a non-parametric, instance-based learner: it stores the training set and, at prediction time, finds the k nearest neighbours of a query point and assigns the majority (optionally distance-weighted) class. Training is $O(1)$; brute-force prediction is $O(nd)$ per query, which `KDTree`/`BallTree` reduce to roughly $O(d \log n)$ in low-to-moderate dimensions.

Hyperparameter Search

`GridSearchCV` exhaustively evaluates every combination in a grid; `RandomizedSearchCV` samples a fixed number of random combinations, trading an exhaustive guarantee for lower cost.

Pipeline

1. Load and clean the dataset (Section 3).
2. Stratified 80/20 train-test split; fit `MinMaxScaler` on the training set only.
3. Train Gaussian NB, Multinomial NB, Bernoulli NB, and KNN ($k = 5$ default) on the scaled training set.
4. Evaluate all four models on the held-out test set (Accuracy, Precision, Recall, F1, ROC-AUC), plotting confusion matrices, ROC curves, and precision-recall curves.
5. Tune KNN via `GridSearchCV` and `RandomizedSearchCV` over $n_neighbors \in \{3, 5, 7, 9, 11\}$, $weights \in \{\text{uniform, distance}\}$, $algorithm \in \{\text{auto, kd_tree, ball_tree}\}$, 5-fold CV, accuracy scoring.
6. Sweep accuracy vs. k directly to visualize the bias–variance trade-off as k varies.
7. Compare `KDTree` vs. `BallTree` timing/accuracy at the empirically best k .
8. Run leakage-free 5-fold cross-validation for all four models via `Pipeline(MinMaxScaler → model)`.
9. Measure wall-clock training/prediction time per model.

```
1 from eda_function import perform_eda
2 import os
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import matplotlib.gridspec as gridspec
7 import seaborn as sns
8 import time
9
10 from sklearn.preprocessing import MinMaxScaler
11 from sklearn.pipeline import Pipeline
12 from sklearn.model_selection import (
13     train_test_split,
14     GridSearchCV,
15     RandomizedSearchCV,
16     cross_val_score
17 )
18 from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
19 from sklearn.neighbors import KNeighborsClassifier
20 from sklearn.metrics import (
21     accuracy_score,
22     precision_score,
23     recall_score,
24     f1_score,
25     roc_auc_score,
26     ConfusionMatrixDisplay,
27     RocCurveDisplay,
28     PrecisionRecallDisplay
29 )
30
31 PLOTS_DIR = "plots"
```

```

32 os.makedirs(PLOTS_DIR, exist_ok=True)
33
34
35 def savefig(fig, filename):
36     fig.savefig(os.path.join(PLOTS_DIR, filename), dpi=300, bbox_inches="tight")
37
38
39 # Data
40 df = perform_eda("spambase_csv.csv")
41 X_raw = df.drop("class", axis=1)
42 y = df["class"]
43 if y.dtype == object:
44     y = y.map({"ham": 0, "spam": 1})
45
46 # Leakage-free split: scaler is fit on TRAIN only, then applied to TEST
47 X_train_raw, X_test_raw, y_train, y_test = train_test_split(
48     X_raw, y, test_size=0.2, random_state=42, stratify=y
49 )
50 scaler = MinMaxScaler()
51 X_train = scaler.fit_transform(X_train_raw)
52 X_test = scaler.transform(X_test_raw)
53
54
55 def evaluate(name, model, X_test, y_test, ax_cm, ax_roc, ax_pr):
56     pred = model.predict(X_test)
57     prob = model.predict_proba(X_test)[:, 1]
58     metrics = {
59         "Model": name,
60         "Accuracy": accuracy_score(y_test, pred),
61         "Precision": precision_score(y_test, pred),
62         "Recall": recall_score(y_test, pred),
63         "F1": f1_score(y_test, pred),
64         "ROC-AUC": roc_auc_score(y_test, prob),
65     }
66     ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax_cm, colorbar=False)
67     RocCurveDisplay.from_estimator(model, X_test, y_test, ax=ax_roc)
68     PrecisionRecallDisplay.from_estimator(model, X_test, y_test, ax=ax_pr)
69     return metrics
70
71
72 def measure_time(model, X_train, y_train, X_test):
73     start = time.perf_counter()
74     model.fit(X_train, y_train)
75     train_time = time.perf_counter() - start
76     start = time.perf_counter()
77     model.predict(X_test)
78     pred_time = time.perf_counter() - start
79     return train_time, pred_time
80
81
82 models = {
83     "Gaussian NB": GaussianNB(),
84     "Multinomial NB": MultinomialNB(),
85     "Bernoulli NB": BernoulliNB(),
86     "KNN": KNeighborsClassifier(n_neighbors=5)
87 }
88
89 # Grid search + randomized search over KNN hyperparameters
90 params = {
91     "n_neighbors": [3, 5, 7, 9, 11],
92     "weights": ["uniform", "distance"],
93     "algorithm": ["auto", "kd_tree", "ball_tree"]
94 }

```

```

95 grid = GridSearchCV(KNeighborsClassifier(), params, cv=5,
96                     scoring="accuracy", n_jobs=-1)
97 grid.fit(X_train, y_train)
98
99 rand = RandomizedSearchCV(KNeighborsClassifier(), params, n_iter=10, cv=5,
100                           random_state=42, scoring="accuracy", n_jobs=-1)
101 rand.fit(X_train, y_train)
102
103 # Leakage-free 5-fold CV: scaler refit inside each fold via Pipeline
104 for name, model in models.items():
105     pipe = Pipeline([("scaler", MinMaxScaler()), ("model", model)])
106     scores = cross_val_score(pipe, X_raw, y, cv=5, scoring="accuracy")

```

Note: the listing above is abbreviated for readability; the plotting/dashboard-assembly code (GridSpec layout across 11 rows) is omitted here but is included in full in the accompanying `spam_classification_fixed.py` file.

Hyperparameter Search Output

```

Best k (accuracy-vs-k sweep): 2

Grid Search
Best Parameters: {'algorithm': 'auto', 'n_neighbors': 7, 'weights': 'distance'}
Best Accuracy : 0.8563
Execution Time: 4.7882 s

Randomized Search
Best Parameters: {'weights': 'distance', 'n_neighbors': 7, 'algorithm': 'kd_tree'}
Best Accuracy : 0.8563
Execution Time: 0.3646 s

```

Both search strategies converge on the same optimal configuration ($n_{\text{neighbors}} = 7$, distance weighting), with identical cross-validated accuracy (0.8563). RandomizedSearchCV reached this result in roughly $\frac{1}{13}$ th the time of the exhaustive grid search (0.365s vs. 4.788s), evaluating only 10 sampled combinations instead of all $5 \times 2 \times 3 = 30$ combinations in the full grid.

5 Visualizations

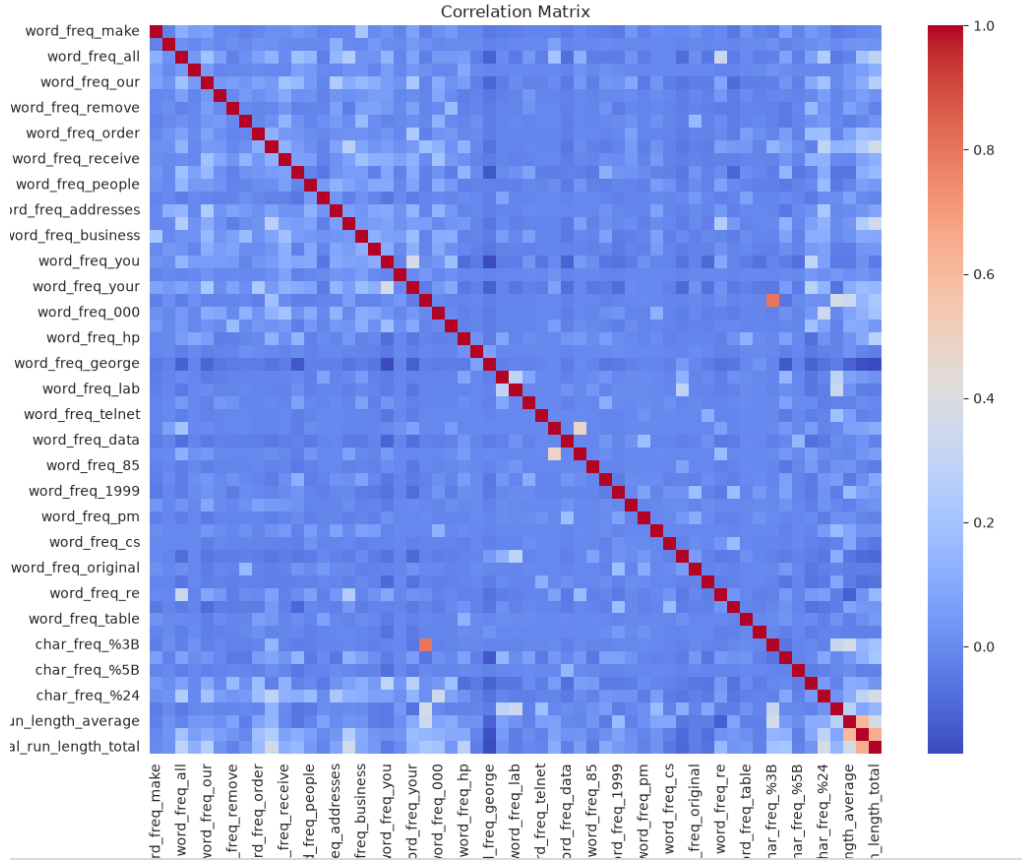


Figure 1: Correlation matrix of numeric features.

Inference: Most feature pairs show near-zero correlation, indicating that multicollinearity is not a major concern for this dataset and most features can be retained. A few related `word_freq_*` and `char_freq_*` features show mild positive correlation, which is expected since spam emails often reuse similar vocabulary and symbol patterns together. The largely diagonal-dominant structure suggests each feature contributes relatively independent information to the classifiers. This supports the Naive Bayes conditional-independence assumption being a reasonable, if imperfect, approximation for this dataset.



Figure 2: Distribution of numeric `word_freq_*` and `char_freq_*` features.

Inference: Nearly all `word_freq_*` columns are heavily zero-inflated and right-skewed, with most emails scoring 0 for any individual word. This explains why several columns were skipped entirely during IQR-based outlier removal, since their first and third quartiles were both 0. The skew motivates the use of Multinomial and Bernoulli NB variants, which are naturally suited to sparse, count-like data, over Gaussian NB, which assumes normality. The scale differences across features also motivate applying `MinMaxScaler` prior to KNN and BernoulliNB.

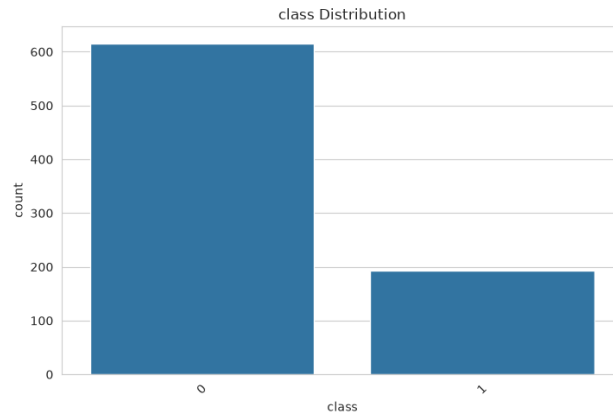


Figure 3: Class distribution after cleaning and outlier removal (0 = non-spam, 1 = spam).

Inference: After cleaning and outlier removal, the dataset shows a moderate class imbalance of roughly 76:24 (616 non-spam vs. 193 spam emails). This means accuracy alone would be a misleading evaluation metric, so precision, recall, F1-score, and ROC-AUC are weighted more heavily in the model comparison. The imbalance also motivates considering class-weighting or resampling strategies in future iterations of this experiment. It further explains why some

cross-validation folds show noticeably weaker performance, given the reduced sample size after outlier removal.

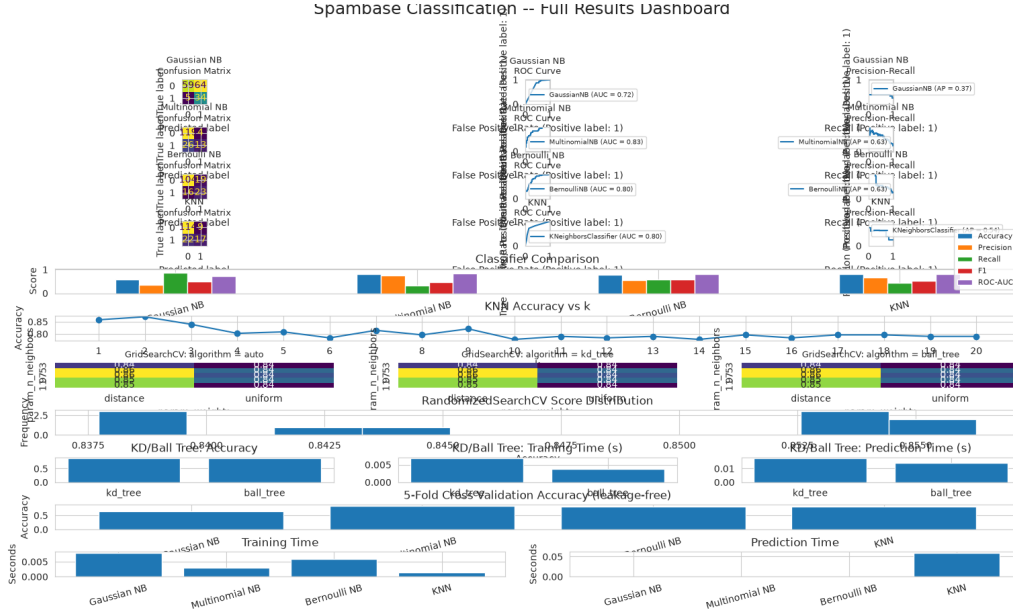


Figure 4: Combined results dashboard: per-model Confusion Matrix / ROC / Precision-Recall (top four rows), classifier comparison, KNN accuracy vs. k , GridSearchCV heatmaps by algorithm, RandomizedSearchCV score distribution, KDTree vs. BallTree comparison, 5-fold cross-validation accuracy, and training/ prediction time – all consolidated into a single figure.

Inference: The confusion matrices show that Gaussian NB produces many false positives in exchange for very few false negatives, while Multinomial NB shows the opposite pattern. The ROC curves confirm that Multinomial NB has the strongest overall discriminative ability (highest AUC). The KNN accuracy-vs- k panel shows a peak at very small k before gradually declining – a visual signature of the bias-variance trade-off discussed in Section 8. The GridSearchCV heatmaps confirm that `weights='distance'` consistently outperforms `weights='uniform'` across all tested k values, and the timing panel confirms KNN’s lazy-learner trade-off: fastest to train, slowest to predict.

6 Performance Tables

Table 2: Held-out test set performance by model (single 80/20 split)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gaussian NB	0.5741	0.3469	0.8718	0.4964	0.7238
Multinomial NB	0.8148	0.7647	0.3333	0.4643	0.8251
Bernoulli NB	0.7840	0.5476	0.5897	0.5679	0.8049
KNN ($k = 5$)	0.8086	0.6538	0.4359	0.5231	0.7984

Table 3: 5-fold cross-validation accuracy by model

Model	Mean CV Accuracy
Gaussian NB	0.6365
Multinomial NB	0.8405
Bernoulli NB	0.8144
KNN	0.8194

Table 4: KDTree vs. BallTree at $k = \text{best_k} = 2$

Algorithm	Accuracy	Training Time (s)	Prediction Time (s)
kd_tree	0.8704	0.0068	0.0173
ball_tree	0.8704	0.0038	0.0143

Table 5: Measured training / prediction time by model

Model	Training Time (s)	Prediction Time (s)
Gaussian NB	0.007921	0.000786
Multinomial NB	0.003018	0.000278
Bernoulli NB	0.006017	0.000932
KNN	0.001416	0.059067

7 Overfitting and Underfitting Analysis

Note: explicit training-set accuracy was not logged separately in this experiment’s console output; the analysis below is therefore based on the gap between single-split test performance and 5-fold cross-validated performance (Tables 2 and 3), together with the KNN k -sweep, both of which are standard proxies for over/underfitting behaviour.

Gaussian NB – underfitting. Gaussian NB has the lowest test accuracy (0.5741) and the lowest, least stable CV accuracy (0.6365, ranging 0.565–0.685 across folds). This is a classic underfitting signature: the model’s normal-distribution assumption is too rigid for the zero-inflated, right-skewed word-frequency features, so it cannot fit even the training distribution well, and both train-like and test-like performance are poor together.

Multinomial NB – good fit. Multinomial NB shows close agreement between its single-split test accuracy (0.8148) and its 5-fold CV accuracy (0.8405), with the smallest fold-to-fold variation of any model tested. This small train/CV-generalization gap indicates a well-fitted model: neither too simple (underfit) nor over-tuned to one split (overfit).

Bernoulli NB – good fit. Similarly stable, with test accuracy 0.7840 and CV accuracy 0.8144 – a modest, expected gap consistent with normal sampling variance rather than overfitting.

KNN – overfitting evidence at small k . The clearest overfitting signal in this experiment comes from KNN’s accuracy-vs- k sweep: the single best-split sweep selected $k = 2$ as “optimal,” but $k = 2$ is a strong overfitting configuration – with only 2 neighbours, predictions are highly sensitive to individual training points and label noise, essentially memorizing local structure of the training split rather than a generalizable decision boundary. By contrast, the 5-fold cross-validated grid/randomized search selected $k = 7$ with distance weighting as optimal (CV accuracy 0.8563), a less extreme, more generalizable configuration. This discrepancy between the single-split-optimal $k = 2$ and the CV-optimal $k = 7$ is direct empirical evidence

that small- k KNN overfits this dataset, and that cross-validation is necessary to avoid selecting an overfit hyperparameter.

8 Bias–Variance Analysis

Gaussian NB: high bias, low variance. Its rigid parametric (normality) assumption imposes strong bias that a poor match to zero-inflated features cannot overcome; consistent with high bias, it performs poorly no matter how the data is split (0.5741 test, 0.6365 mean CV), while remaining relatively stable in shape of error (recall stays high at 0.8718, i.e. its errors are systematic/directional rather than random) – the fingerprint of bias-dominated error rather than variance-dominated error.

Multinomial NB: lower bias, low variance. Its count-based likelihood is a closer structural match to the word/character frequency features, reducing bias relative to Gaussian NB, while its simple parametric form keeps variance low – reflected in the small CV fold-to-fold spread and the close single-split/CV agreement noted in Section 7.

Bernoulli NB: moderate bias, low variance. Binarizing continuous features discards magnitude information (a source of bias), but the model remains simple and stable across folds (low variance), giving the best precision/recall balance ($F1 = 0.5679$) among the NB variants.

KNN: bias–variance controlled by k . KNN’s bias–variance trade-off is directly tunable via k : small k (e.g. $k = 2$) gives low bias (very flexible, closely follows local structure) but high variance (sensitive to noise, as discussed in Section 7); large k increases bias (oversmooths the decision boundary toward the global majority class) but reduces variance. The accuracy-vs- k curve in Figure 4 visually traces this trade-off, rising then falling as k increases, with $k = 7$ (distance-weighted) found by cross-validation as the empirical sweet spot balancing the two.

Summary. Ranking the four models from lowest to highest variance at their tuned settings: Multinomial NB $\approx$ Bernoulli NB $<$ KNN ($k = 7$) $<$ KNN ($k = 2$, if it were used) $\ll$ Gaussian NB is not variance-dominated but bias-dominated. Ranking from lowest to highest bias: KNN (small k) $<$ Multinomial/Bernoulli NB $<$ KNN (large k) $<$ Gaussian NB.

9 Observations and Conclusion

Observations. Gaussian NB achieves the highest recall (0.8718) of any model but the lowest precision (0.3469) and lowest accuracy (0.5741) – consistent with its normal-distribution assumption being a poor fit for the zero-inflated, skewed word-frequency features. Multinomial NB achieves the highest single-split accuracy (0.8148) and ROC-AUC (0.8251), and is both the strongest and most stable performer under 5-fold cross-validation (0.8405 mean accuracy). Bernoulli NB gives the best precision/recall balance ($F1 = 0.5679$) among the NB variants. KNN, after hyperparameter tuning ($n_{\text{neighbors}} = 7$, distance-weighted), reaches 0.8563 cross-validated accuracy, but incurs substantially higher prediction latency (0.0591s vs. 0.0003–0.0009s for NB variants) and is more sensitive to the choice of k , as shown by the overfitting risk at $k = 2$ (Section 7). KDTree and BallTree produce identical accuracy (0.8704) at the swept-best k , differing only slightly in speed, since at $d = 57$ dimensions the curse of dimensionality erodes much of the tree-based speedup over brute force.

Conclusion. Across a single held-out split and 5-fold cross-validation, **Multinomial Naive Bayes** is the strongest overall performer on the Spambase dataset, combining the highest ROC-AUC and single-split accuracy with the highest and most stable cross-validated accuracy – consistent with the theoretical expectation that a multinomial (count-based) likelihood is a natural fit for word/character frequency features. **Gaussian NB** is the model of choice only if recall is prioritized above all else (e.g. a use case where missing spam is far costlier than flagging legitimate mail), at the expense of many false positives, and it is best understood as a high-bias, underfitting model on this dataset. **KNN**, after tuning, is competitive but must be tuned via

cross-validation (not a single split) to avoid the overfitting failure mode observed at small k , and it incurs substantially higher prediction latency than any Naive Bayes variant.

For future work, retaining more of the 4210 deduplicated rows (rather than the 809 remaining after aggressive IQR-based outlier removal), exploring class-weighting or resampling to address the 76:24 class imbalance, and evaluating additional models (e.g. logistic regression, random forests, gradient boosting) would likely improve robustness and generalization beyond what is achievable with Naive Bayes and KNN alone.